



GuriMAX: datos confiables y arquitectura evolutiva

Base de datos, validación SQL y Clean Architecture

Sistema POS e inventario para Provisiones Gurima

Manus AI · 27 de agosto de 2026



**El negocio necesita una fuente única de
verdad**

La arquitectura mantiene a MySQL detrás de la API



Regla de seguridad

Las PCs no reciben credenciales de MySQL ni acceden directamente al puerto de datos.

Stack tecnológico

Go · Wails · Svelte · TypeScript · MySQL · database/sql · sqlc · migraciones SQL.[1]

| El esquema sigue los límites del negocio

Capacidad	Tablas representativas
Identidad	users, roles, permissions, sessions
Catálogo	products, categories, units, product_barcodes, product_lots
POS y caja	sales, sale_lines, sale_payments, cash_sessions, cash_movements
Inventario	inventory_balances, stock_movements, inventory_counts
Compras	purchases, purchase_receipts, purchase_incidents, payables
Crédito y devoluciones	customers, credit_accounts, credit_entries, sales_returns, refunds
Control operativo	billing_documents, audit_log, idempotency_keys, backup_jobs

Decisión: monolito modular; módulos separados internamente, pero un solo despliegue en la primera versión.

Las restricciones convierten reglas en garantías

DINERO

Columnas `*_cents` y moneda `DOP`; no se usa `float64` para importes.

INTEGRIDAD

Claves foráneas, claves únicas, `CHECK`, estados explícitos e índices.

TRAZABILIDAD

Movimientos de inventario, caja y crédito; auditoría con actor, motivo y valores.

REINTENTOS

`idempotency_key` para evitar duplicar ventas, recepciones, abonos o devoluciones.

EJEMPLO

Una única sesión activa por caja y fecha se garantiza mediante una columna generada y una clave única.

catalog.sql ya cubre las lecturas críticas

Consulta	Cobertura	Evaluación
GetProductByBarcode	Producto activo, categoría, unidad, precios, impuestos y stock disponible.	Correcta para POS.
SearchProducts	Nombre, SKU, código, categoría, estado activo y paginación.	Correcta para búsqueda.
ListLowStockProducts	Existencia disponible frente al mínimo configurado.	Correcta para alerta.
GetExpiringLots	Lotes con existencia de productos perecederos activos dentro de un rango.	Correcta para vencimiento.

LÍMITE

Leer disponibilidad no reserva existencias ni decide si se permite stock negativo.

DETALLE DE INTEGRACIÓN

sqlc genera cantidades DECIMAL como string/sql.NullString; el repositorio debe convertirlas a objetos de valor.

sales.sql cubre detalle, cobros y reportes básicos

Consulta

Cobertura

GetSaleByID

Totales, cliente, caja, fecha de negocio, usuario, cancelación e idempotencia.

GetSaleByIdempotencyKey

Detección de reintentos de la misma operación.

ListSaleLines

Instantánea del producto, lote, cantidad, precio, descuento e impuesto.

ListSalePayments

Pagos mixtos, estado, referencia externa y verificación.

ListSalesByDate

Período, estado, sesión, usuario, desglose financiero y paginación.

Límite: una consulta de lectura no puede validar por sí sola caja, crédito, permisos, pagos compatibles ni inventario.

La lectura no sustituye al caso de uso

REGLA

CAPA RESPONSABLE

Totales, descuentos e impuestos

Dominio y política de precios/impuestos.

Límite y bloqueo de crédito

Agregado `CustomerAccount` .

Reserva y salida de inventario

Gateway de inventario dentro de transacción.

Pagos y caja activa

Gateway de caja y caso de uso.

Aprobación de acciones sensibles

Autorización contextual y auditoría.

Principio: SQL devuelve datos; la aplicación coordina; el dominio decide.

Repositorios: traducir sin filtrar infraestructura

APPLICATION/PORTS

ProductReader · SaleReader · SaleWriter

INTERFACES ▲

INFRASTRUCTURE/MYSQL

CatalogRepository · SalesRepository

▼ IMPLEMENTACIÓN

PERSISTENCIA

sqlc.Queries / database/sql

Los puertos viven en `application/ports`.

Las implementaciones concretas viven en `infrastructure/mysql`.

Los mapeadores convierten IDs, dinero, cantidades y nulos.

El dominio nunca recibe `sqlc.Row`,
`sql.NullString` ni `*sql.Tx`.

Los casos de uso coordinan módulos, no tablas

ORDEN RECOMENDADO

1. `Login` y autorización.
2. `SearchProducts` y catálogo.
3. `OpenCashSession`.
4. `CompleteSale`.
5. `ReceivePurchase`.
6. Ajustes, crédito, devoluciones y cierre.

Puertos entre módulos:

`InventoryGateway`, `CashGateway`, `CreditGateway`, `AuditWriter` y `FiscalDocumentProvider`.

Ventaja: se pueden cambiar MySQL, HTTP o el cliente sin mover las reglas del negocio.

CompleteSale debe ser una sola transacción

AUTORIZAR USUARIO

COMPROBAR IDEMPOTENCIA

BLOQUEAR CAJA E INVENT...

RECALCULAR PRECIOS E IM...

VENTA: LÍNEAS + PAGOS

INVENTARIO, CRÉDITO Y CO...

COMMIT O ROLLBACK

GARANTÍA

Si falla inventario, caja, crédito, comprobante o auditoría, la venta completa se revierte.

IMPLEMENTACIÓN

`TransactionManager / UnitOfWork`
y `sqlc.New(tx)` o `WithTx`.

La base está lista para la siguiente capa

LISTO

Migración reversible y esquema modular

Compose local con healthcheck

Consultas de lectura validadas

Código Go generado por sqlc

Idempotencia y auditoría modeladas

PRÓXIMO INCREMENTO

Conexión segura en `internal/platform/mysql`

Comandos sqlc de escritura

Repositorios con mapeadores explícitos

Casos de uso y autorización contextual

Pruebas de integración con MySQL

SECUENCIA: identidad → catálogo/inventario → POS/caja → compras → crédito/devoluciones → facturación/reportes.

Cierre: las consultas cubren las lecturas esenciales; las reglas críticas deben ejecutarse en dominio y aplicación, con repositorios detrás de puertos y transacciones que protejan toda la operación.